DYNAMIC PRICE AI

AI-Powered Dynamic Pricing Engine

Project Demo Documentation

Requirements | Architecture | Technologies | Implementation

Version 1.0 | 2025

Table of Contents

Table of Contents 2
1. Project Overview 4
  1.1 Objectives 4
  1.2 Scope 4
2. Functional Requirements 5
  2.1 Core Pricing Engine 5
  2.2 Data Ingestion & Management 5
  2.3 Machine Learning Model 5
  2.4 API & Integration Layer 5
  2.5 Dashboard & Reporting 6
3. Non-Functional Requirements 7
  3.1 Performance 7
  3.2 Scalability 7
  3.3 Availability & Reliability 7
  3.4 Security 7
  3.5 Compliance & Auditability 7
4. System Architecture 8
  4.1 Architecture Layers 8
  4.2 Data Flow 8
5. Technology Stack 9
  5.1 Backend & API 9
  5.2 Machine Learning & AI 9
  5.3 Data Engineering 9
  5.4 Database & Storage 10
  5.5 Frontend 10
  5.6 Infrastructure & DevOps 10
6. Step-by-Step Implementation Guide 12
  Phase 1 — Data Foundation (Weeks 1–4) 12
    Step 1: Environment Setup 12
    Step 2: Data Ingestion Pipeline 12
  Phase 2 — ML Model Development (Weeks 5–10) 12
    Step 3: Feature Engineering 12
    Step 4: Model Training & Selection 12
    Step 5: Model Serving 13

Phase 3 — API & Integration (Weeks 11–14) 13
Step 6: Build the Pricing API 13
Step 7: External System Integration 13
Phase 4 — Dashboard & UI (Weeks 15–18) 13
Step 8: Build the Pricing Dashboard 13
Phase 5 — Testing & Go-Live (Weeks 19–22) 14
Step 9: Quality Assurance 14
Step 10: Production Deployment & Monitoring 14
7. API Reference 15
7.1 Key Endpoints 15
8. Risks & Mitigations 16
9. Project Timeline 17
10. Glossary 18

1. Project Overview

Dynamic Price AI is an intelligent, real-time pricing engine that leverages machine learning and AI algorithms to automatically adjust product or service prices based on market conditions, competitor pricing, demand signals, inventory levels, and customer behavior. The system is designed to maximize revenue and profitability while maintaining competitive positioning.

1.1 Objectives

- Automate pricing decisions using AI/ML models to eliminate manual pricing overhead.
- Maximize revenue by dynamically responding to real-time demand, supply, and competitor data.
- Provide a transparent and explainable pricing dashboard for business stakeholders.
- Ensure scalability to handle millions of SKUs and transactions simultaneously.
- Enable A/B testing of pricing strategies with performance analytics.

1.2 Scope

The project covers end-to-end design and development of a dynamic pricing platform including data ingestion pipelines, ML model training and inference, API layer, and a business intelligence dashboard.

2. Functional Requirements

2.1 Core Pricing Engine

1. The system shall ingest real-time data streams including competitor prices, demand signals, inventory levels, and historical sales.
2. The engine shall compute optimal prices for each SKU at configurable intervals (e.g., every 5 minutes, hourly, or daily).
3. Price recommendations shall be bounded by configurable floor and ceiling rules to prevent extreme adjustments.
4. The system shall support multiple pricing strategies: cost-plus, value-based, competitive, and demand-based pricing.
5. Price changes shall be logged with full audit trails including timestamp, model version, and input features.

2.2 Data Ingestion & Management

6. The system shall integrate with external competitor price APIs (e.g., web scrapers, partner feeds).
7. Historical sales and pricing data shall be ingested from ERP/CRM/POS systems via secure connectors.
8. The platform shall support batch and streaming data ingestion pipelines.
9. Data validation and anomaly detection shall be applied at the ingestion layer.

2.3 Machine Learning Model

10. The ML pipeline shall train, validate, and deploy regression and reinforcement learning models for price optimization.
11. The system shall support automated model retraining on a scheduled basis (weekly/monthly) or when model drift is detected.
12. Feature engineering shall include: time-of-day, day-of-week, seasonality, demand elasticity, inventory velocity, and competitor price deltas.
13. Model explainability (SHAP values) shall be available for each pricing decision.

2.4 API & Integration Layer

14. A RESTful API shall expose pricing endpoints to upstream e-commerce platforms, POS systems, and ERP.
15. The API shall return price recommendations within 200ms for 95th percentile of requests.
16. Webhook support shall be provided for event-driven price updates.
17. Authentication via OAuth 2.0 and API key management shall be enforced.

2.5 Dashboard & Reporting

18. A web-based dashboard shall visualize current prices, price history, revenue impact, and competitor comparisons.
19. Business users shall be able to set pricing rules, overrides, and approval workflows without engineering support.
20. The system shall generate automated daily and weekly pricing performance reports.
21. A/B test management UI shall allow users to create, monitor, and conclude pricing experiments.

3. Non-Functional Requirements

3.1 Performance

- API response time: < 200ms at p95 under normal load.
- System shall handle up to 10,000 concurrent pricing requests per second.
- Batch processing of 1 million SKU price updates within 30 minutes.

3.2 Scalability

- Horizontally scalable microservices deployed on Kubernetes.
- Database partitioning strategy to support 100M+ product records.
- Auto-scaling policies based on CPU utilization and request queue depth.

3.3 Availability & Reliability

- System uptime SLA: 99.9% (less than 8.7 hours downtime per year).
- Multi-zone deployment for high availability and disaster recovery.
- Circuit breaker patterns implemented for all external API dependencies.
- Automated health checks and alerting via monitoring stack.

3.4 Security

- All data in transit encrypted with TLS 1.3.
- All data at rest encrypted with AES-256.
- Role-Based Access Control (RBAC) for dashboard and API access.
- PII and sensitive pricing data masked in logs.
- Regular security audits and penetration testing.

3.5 Compliance & Auditability

- Full audit log of every pricing decision with model version, input features, and output.
- Pricing decisions shall be explainable and traceable for regulatory purposes.
- Data retention policies compliant with GDPR and local regulations.

4. System Architecture

The Dynamic Price AI platform follows a microservices architecture with four primary layers: Data Layer, ML Platform, API Gateway, and Presentation Layer. Each layer is independently deployable and communicates via event-driven messaging and REST APIs.

4.1 Architecture Layers

Layer	Components	Responsibility
Data Layer	Kafka, Spark, PostgreSQL, Redis	Data ingestion, storage, streaming
ML Platform	MLflow, scikit-learn, PyTorch, Ray	Model training, serving, monitoring
API Gateway	FastAPI, Kong Gateway, Redis Cache	Request routing, auth, caching
Presentation	React, Next.js, Chart.js	Dashboard, reporting, A/B testing UI
Infrastructure	Kubernetes, Docker, Terraform, AWS/GCP	Orchestration, CI/CD, cloud hosting

4.2 Data Flow

22. External data sources (competitor feeds, sales systems, inventory APIs) push data to Kafka topics.
23. Spark Streaming consumers process and transform raw data in real-time.
24. Processed features are stored in PostgreSQL (persistent) and Redis (low-latency feature store).
25. The ML inference service reads features from Redis and runs the pricing model.
26. Price recommendations are returned via the FastAPI gateway and cached in Redis.
27. All pricing decisions are asynchronously logged to the audit database.
28. The React dashboard polls the API and displays live pricing metrics.

5. Technology Stack

5.1 Backend & API

Technology	Version	Purpose
Python	3.11+	Core backend language for ML and API services
FastAPI	0.110+	High-performance async REST API framework
Node.js / Express	20 LTS	Webhook service and real-time notification server
Kong Gateway	3.x	API gateway, rate limiting, auth plugin management
Celery + Redis	5.x	Async task queue for batch pricing jobs

5.2 Machine Learning & AI

Technology	Version	Purpose
scikit-learn	1.4+	Regression, gradient boosting for price prediction
PyTorch	2.2+	Deep learning models for demand forecasting
MLflow	2.x	Experiment tracking, model registry, deployment
Ray / RLlib	2.x	Reinforcement learning for long-horizon pricing
SHAP	0.44+	Model explainability and feature importance
Optuna	3.x	Hyperparameter optimization for ML models
Evidently AI	0.4+	Model drift detection and data quality monitoring

5.3 Data Engineering

Technology	Version	Purpose
Apache Kafka	3.7+	Real-time event streaming and data ingestion
Apache Spark	3.5+	Batch and streaming data transformation
Apache Airflow	2.9+	Workflow orchestration for ETL and model retraining

dbt	1.8+	Data transformations and modeling in the warehouse
Great Expectations	0.18+	Data validation and quality checks

5.4 Database & Storage

Technology	Version	Purpose
PostgreSQL	16+	Primary relational DB for pricing rules, audit logs
Redis	7.x	Feature store, session cache, real-time price cache
ClickHouse	24.x	Analytics data warehouse for pricing metrics and BI
Amazon S3 / GCS	Latest	Raw data lake, model artifacts, historical exports
Elasticsearch	8.x	Full-text search for product catalog and pricing history

5.5 Frontend

Technology	Version	Purpose
React	18+	Core UI library for the pricing dashboard
Next.js	14+	SSR/SSG framework with API routes
TypeScript	5.x	Type-safe frontend development
Recharts / Chart.js	Latest	Price trend charts and analytics visualizations
TailwindCSS	3.x	Utility-first responsive UI styling
React Query	5.x	Server state management and real-time data fetching

5.6 Infrastructure & DevOps

Technology	Version	Purpose
Docker	25+	Containerization of all microservices
Kubernetes (EKS/GKE)	1.29+	Container orchestration and auto-scaling

Terraform	1.8+	Infrastructure as Code (IaC) for cloud provisioning
GitHub Actions	Latest	CI/CD pipelines for build, test, and deploy
Prometheus + Grafana	Latest	Metrics collection, alerting, and visualization
ELK Stack	8.x	Centralized log management and search
Istio	1.21+	Service mesh for mTLS, traffic management, observability

6. Step-by-Step Implementation Guide

Phase 1 — Data Foundation (Weeks 1–4)

Step 1: Environment Setup

29. Provision cloud infrastructure using Terraform scripts (VPC, subnets, IAM roles, S3/GCS buckets).
30. Deploy Kubernetes cluster (EKS/GKE) with node groups for API, ML, and data workloads.
31. Set up Docker registry (ECR/GCR) and configure GitHub Actions CI/CD pipelines.
32. Install and configure Kafka cluster with topics: raw-prices, competitor-data, inventory-events, sales-events.
33. Deploy PostgreSQL (primary + read replica) and Redis cluster.

Step 2: Data Ingestion Pipeline

34. Develop Kafka producers for each data source (competitor scrapers, ERP connectors, POS feeds).
35. Build Spark Streaming jobs to consume Kafka topics, validate schemas, and write to the data lake.
36. Implement Airflow DAGs to schedule batch ingestion jobs and data quality checks.
37. Configure Great Expectations suite for data validation and anomaly alerts.
38. Validate end-to-end data flow from source to PostgreSQL with 100% schema compliance.

Phase 2 — ML Model Development (Weeks 5–10)

Step 3: Feature Engineering

39. Extract and transform pricing features: historical price elasticity, demand velocity, competitor delta, seasonality indices.
40. Build feature pipelines using Spark and store output in the Redis feature store.
41. Create feature documentation and versioning in MLflow Feature Registry.
42. Validate feature distributions and monitor for data drift using Evidently AI.

Step 4: Model Training & Selection

43. Train baseline models: Linear Regression, Gradient Boosting (XGBoost/LightGBM), and Neural Network.
44. Run hyperparameter optimization using Optuna with cross-validation.

45. Evaluate models on held-out test sets using revenue uplift as the primary business metric.
46. Register the best-performing model in MLflow Model Registry with staging/production tags.
47. Generate SHAP explainability reports for the selected model.

Step 5: Model Serving

48. Package the model as a FastAPI inference service with Dockerized deployment.
49. Configure auto-scaling on Kubernetes based on request queue depth.
50. Implement shadow mode deployment to compare new model vs current model in production traffic.
51. Set up model drift alerts using Evidently AI with automated retraining triggers.

Phase 3 — API & Integration (Weeks 11–14)

Step 6: Build the Pricing API

52. Develop FastAPI endpoints: GET /price/{sku_id}, POST /price/batch, POST /price/override.
53. Integrate Redis caching layer with 5-second TTL for high-frequency SKUs.
54. Configure Kong Gateway with rate limiting (1000 req/min per client), JWT auth, and logging plugins.
55. Implement Circuit Breaker pattern using Hystrix/Resilience4J for all upstream dependencies.
56. Write API documentation with OpenAPI 3.0 and Swagger UI.
57. Conduct load testing with k6 to validate 200ms p95 latency target.

Step 7: External System Integration

58. Build ERP/POS connectors using pre-built adapters (SAP, Oracle, Shopify, WooCommerce).
59. Configure webhooks for event-driven price push to downstream systems.
60. Implement idempotency keys and retry logic for all outbound API calls.
61. Test end-to-end integration with staging ERP/POS instances.

Phase 4 — Dashboard & UI (Weeks 15–18)

Step 8: Build the Pricing Dashboard

62. Design and implement the React/Next.js dashboard with role-based views (Admin, Analyst, Business User).
63. Integrate Recharts for live price trend charts, revenue impact graphs, and competitor comparison tables.

64. Build pricing rule engine UI: floor/ceiling configuration, category-level rules, time-based overrides.
65. Implement the A/B test management module: create experiments, assign traffic splits, view results.
66. Connect dashboard to ClickHouse for analytics queries and Elasticsearch for product search.

Phase 5 — Testing & Go-Live (Weeks 19–22)

Step 9: Quality Assurance

67. Execute unit and integration test suites (target: 85%+ code coverage).
68. Run end-to-end regression tests simulating full pricing update cycles.
69. Conduct security penetration testing and remediate findings.
70. Perform User Acceptance Testing (UAT) with business stakeholders.

Step 10: Production Deployment & Monitoring

71. Deploy to production using blue-green deployment strategy with automated rollback.
72. Configure Prometheus alerting rules for API latency, error rates, and ML model health.
73. Set up Grafana dashboards for engineering (system metrics) and business (revenue impact) teams.
74. Conduct a 2-week hypercare period with on-call engineering support.
75. Deliver knowledge transfer sessions and hand-off documentation to the operations team.

7. API Reference

7.1 Key Endpoints

Method	Endpoint	Auth	Description
GET	/price/{sku_id}	Bearer Token	Get optimal price for a single SKU
POST	/price/batch	Bearer Token	Batch price fetch for up to 1000 SKUs
POST	/price/override	Bearer Token + Role	Manually override price for a SKU with expiry
GET	/price/history/{sku_id}	Bearer Token	Retrieve price history with change reasons
GET	/experiments	Bearer Token	List all active A/B pricing experiments
POST	/experiments	Bearer Token + Admin	Create a new A/B pricing experiment
GET	/health	None	System health check for load balancer probes

8. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Model produces extreme price recommendations	Medium	High	Enforce floor/ceiling guardrails and human approval for >20% changes
Competitor data feed outage	Medium	Medium	Fall back to last-known competitor data with decay weighting
ML model drift over time	High	Medium	Automated weekly drift monitoring with retraining triggers
API latency spikes under peak load	Low	High	Redis caching, auto-scaling, and load testing pre-launch
Regulatory pricing compliance violations	Low	High	Pricing audit logs, legal review of pricing rules, and compliance checks

9. Project Timeline

Phase	Activities	Duration	Key Deliverable
Phase 1	Data Foundation	Weeks 1–4	Live data pipelines, validated data lake
Phase 2	ML Development	Weeks 5–10	Trained & deployed pricing model
Phase 3	API & Integration	Weeks 11–14	Production-ready REST API with integrations
Phase 4	Dashboard	Weeks 15–18	Full-featured business dashboard
Phase 5	QA & Go-Live	Weeks 19–22	Production deployment + hypercare

10. Glossary

Term	Definition
Dynamic Pricing	A strategy where prices are adjusted in real-time based on market factors and demand signals.
SKU	Stock Keeping Unit — a unique identifier for a product or service.
Price Elasticity	A measure of how sensitive customer demand is to price changes.
MLflow	An open-source ML platform for tracking experiments, packaging models, and deploying to production.
SHAP	SHapley Additive exPlanations — a method to explain individual predictions made by ML models.
Feature Store	A centralized repository for storing and serving pre-computed ML features in real-time.
A/B Testing	A controlled experiment to compare two pricing strategies and measure their revenue impact.
Model Drift	Degradation in model accuracy over time due to changes in real-world data distributions.
Circuit Breaker	A design pattern that prevents cascading failures by stopping calls to a failing service.

End of Document | Dynamic Price AI | Demo Documentation | 2025